

Theory of Computation

Lecture 2: Languages, Problems, and the Idea of Computation

Sheikh Azizul Hakim

Lecturer, Department of CSE, BUET

Last compiled: June 21, 2026 8:07pm +06:00

Everything that a computer processes can be represented as strings.

Everything.

Is This Data?

```
42  
01101010  
if (x > 5)
```

```
ATTGCCGA  
<emoji>  
Write a poem.
```

Is This Data?

```
42  
01101010  
if (x > 5)
```

```
ATTGCCGA  
<emoji>  
Write a poem.
```

What do these have in common?

Is This Data?

```
42  
01101010  
if (x > 5)
```

```
ATTGCCGA  
<emoji>  
Write a poem.
```

What do these have in common?

They are finite sequences of symbols.

Everything Is Strings

Object	String representation
Integer	12345
DNA sequence	ACTGACTG
Program	if (x > 5) { ... }
Image	JPEG/PNG bit stream
Audio	MP3/WAV bit stream
Prompt	“Write a poem.”

Everything Is Strings

Object	String representation
Integer	12345
DNA sequence	ACTGACTG
Program	if (x > 5) { ... }
Image	JPEG/PNG bit stream
Audio	MP3/WAV bit stream
Prompt	“Write a poem.”

Data → **Strings**

Alphabet

An **alphabet** is a finite nonempty set of symbols.

Alphabet

An **alphabet** is a finite nonempty set of symbols.

Examples:

$$\Sigma_1 = \{0, 1\},$$

$$\Sigma_2 = \{a, b, c, \dots, z\},$$

$$\Sigma_3 = \{A, C, G, T\}.$$

Alphabet

An **alphabet** is a finite nonempty set of symbols.

Examples:

$$\Sigma_1 = \{0, 1\},$$

$$\Sigma_2 = \{a, b, c, \dots, z\},$$

$$\Sigma_3 = \{A, C, G, T\}.$$

In computation, we choose an alphabet according to the kind of objects we want to encode.

Strings

Over the alphabet

$$\Sigma = \{0, 1\},$$

examples of strings are:

ϵ , 0, 1, 01, 1011, 111000

Strings

Over the alphabet

$$\Sigma = \{0, 1\},$$

examples of strings are:

ε , 0, 1, 01, 1011, 111000

The **length** of a string is the number of symbols in it:

$$|1011| = 4, \quad |\varepsilon| = 0.$$

Concatenation

$$01 \cdot 10 = 0110.$$

String Operations

Concatenation

$$01 \cdot 10 = 0110.$$

Power

$$0^2 = 00, \quad (ab)^3 = ababab.$$

String Operations

Concatenation

$$01 \cdot 10 = 0110.$$

Power

$$0^2 = 00, \quad (ab)^3 = ababab.$$

Reverse

$$(10110)^R = 01101.$$

String Operations

Concatenation

$$01 \cdot 10 = 0110.$$

Power

$$0^2 = 00, \quad (ab)^3 = ababab.$$

Reverse

$$(10110)^R = 01101.$$

Strings are simple objects, but they can encode almost anything.

All Possible Strings

For an alphabet Σ , the set of all finite strings over Σ is written:

$$\Sigma^*.$$

All Possible Strings

For an alphabet Σ , the set of all finite strings over Σ is written:

$$\Sigma^*.$$

For $\Sigma = \{0, 1\}$,

$$\Sigma^* = \{\varepsilon, 0, 1, 00, 01, 10, 11, 000, \dots\}.$$

All Possible Strings

For an alphabet Σ , the set of all finite strings over Σ is written:

$$\Sigma^*.$$

For $\Sigma = \{0, 1\}$,

$$\Sigma^* = \{\varepsilon, 0, 1, 00, 01, 10, 11, 000, \dots\}.$$

Is Σ^* finite or infinite?

What Is a Language?

A **language** is a set of strings.

What Is a Language?

A **language** is a set of strings.

If $\Sigma = \{0, 1\}$, then each of the following is a language:

$$L_1 = \{0, 10, 111\},$$

$$L_2 = \{w \in \Sigma^* : w \text{ contains an even number of 1s}\},$$

$$L_3 = \Sigma^*,$$

$$L_4 = \emptyset.$$

What Is a Language?

A **language** is a set of strings.

If $\Sigma = \{0, 1\}$, then each of the following is a language:

$$L_1 = \{0, 10, 111\},$$

$$L_2 = \{w \in \Sigma^* : w \text{ contains an even number of 1s}\},$$

$$L_3 = \Sigma^*,$$

$$L_4 = \emptyset.$$

A language separates YES strings from NO strings.

Consider these questions:

- Is 100101 divisible by 3?
- Is `((()()))` balanced?
- Is this Python program syntactically valid?
- Is this Boolean formula satisfiable?

Computational Problems

Consider these questions:

- Is 100101 divisible by 3?
- Is `((()()))` balanced?
- Is this Python program syntactically valid?
- Is this Boolean formula satisfiable?

What do these problems have in common?

Computational Problems

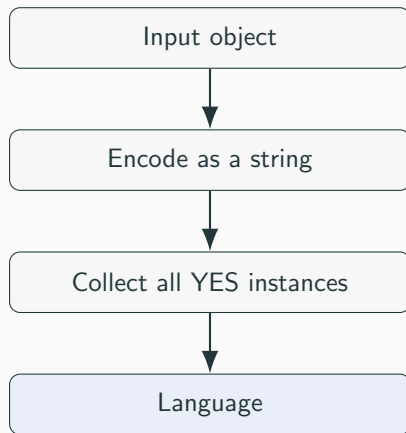
Consider these questions:

- Is 100101 divisible by 3?
- Is `((()()))` balanced?
- Is this Python program syntactically valid?
- Is this Boolean formula satisfiable?

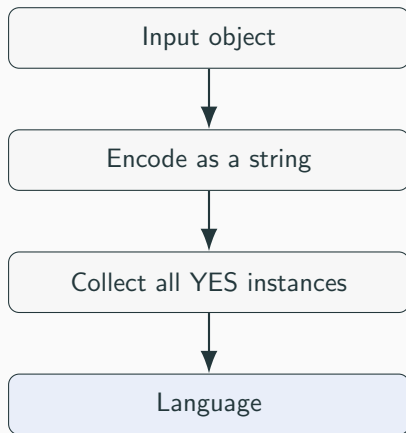
What do these problems have in common?

Each input has a YES or NO answer.

The Central Idea



The Central Idea



A computational problem becomes a language.

Example: Even Number of 1's

Define:

$$L_{\text{even}} = \{w \in \{0, 1\}^* : w \text{ contains an even number of 1s}\}.$$

Example: Even Number of 1's

Define:

$$L_{\text{even}} = \{w \in \{0, 1\}^* : w \text{ contains an even number of 1s}\}.$$

In the language

ε , 00, 11, 1010, 010101

Not in the language

1, 111, 101, 0001000

Example: Even Number of 1's

Define:

$$L_{\text{even}} = \{w \in \{0, 1\}^* : w \text{ contains an even number of 1s}\}.$$

In the language

ε , 00, 11, 1010, 010101

Not in the language

1, 111, 101, 0001000

Soon we will build a tiny machine that recognizes this language.

Example: Palindromes

A palindrome reads the same forward and backward.

Example: Palindromes

A palindrome reads the same forward and backward.

$$\text{PAL} = \{w \in \{0, 1\}^* : w = w^R\}.$$

Example: Palindromes

A palindrome reads the same forward and backward.

$$\text{PAL} = \{w \in \{0, 1\}^* : w = w^R\}.$$

Examples

0, 1, 0110, 1001, 01010

Non-examples

01, 110, 0101, 10010

Example: Palindromes

A palindrome reads the same forward and backward.

$$\text{PAL} = \{w \in \{0, 1\}^* : w = w^R\}.$$

Examples

0, 1, 0110, 1001, 01010

Non-examples

01, 110, 0101, 10010

This language requires more memory than finite automata have.

Example: Valid JSON

An LLM may be asked:

```
Generate a valid JSON object.
```

Example: Valid JSON

An LLM may be asked:

```
Generate a valid JSON object.
```

Valid JSON:

```
{  
  "name": "Aziz",  
  "courses": ["CSE", "EEE"]  
}
```

Example: Valid JSON

An LLM may be asked:

```
Generate a valid JSON object.
```

Valid JSON:

```
{  
  "name": "Aziz",  
  "courses": ["CSE", "EEE"]  
}
```

The corresponding language is:

$$\text{JSON} = \{x : x \text{ is a syntactically valid JSON string}\}.$$

Example: Valid JSON

An LLM may be asked:

```
Generate a valid JSON object.
```

Valid JSON:

```
{  
  "name": "Aziz",  
  "courses": ["CSE", "EEE"]  
}
```

The corresponding language is:

$$\text{JSON} = \{x : x \text{ is a syntactically valid JSON string}\}.$$

Structured generation is language generation.

Example: Valid Python Programs

Question:

What language is recognized by Python's parser?

Example: Valid Python Programs

Question:

What language is recognized by Python's parser?

$\text{PYTHON} = \{x : x \text{ is a syntactically valid Python program}\}.$

Example: Valid Python Programs

Question:

What language is recognized by Python's parser?

$\text{PYTHON} = \{x : x \text{ is a syntactically valid Python program}\}.$

```
def square(x):  
    return x*x
```

Example: Valid Python Programs

Question:

What language is recognized by Python's parser?

$\text{PYTHON} = \{x : x \text{ is a syntactically valid Python program}\}.$

```
def square(x):  
    return x*x
```

Programming languages are literally languages.

Example: SAT

A Boolean formula is **satisfiable** if some assignment of truth values makes it true.

Example: SAT

A Boolean formula is **satisfiable** if some assignment of truth values makes it true.

$$\text{SAT} = \{\varphi : \varphi \text{ is satisfiable}\}.$$

Example: SAT

A Boolean formula is **satisfiable** if some assignment of truth values makes it true.

$$\text{SAT} = \{\varphi : \varphi \text{ is satisfiable}\}.$$

YES instance

$$(x \vee y) \wedge (\neg x \vee z)$$

NO instance

$$x \wedge \neg x$$

Example: SAT

A Boolean formula is **satisfiable** if some assignment of truth values makes it true.

$$\text{SAT} = \{\varphi : \varphi \text{ is satisfiable}\}.$$

YES instance

$$(x \vee y) \wedge (\neg x \vee z)$$

NO instance

$$x \wedge \neg x$$

SAT is one of the most important languages in computer science.

Why Languages?

Without a common abstraction, these look unrelated:

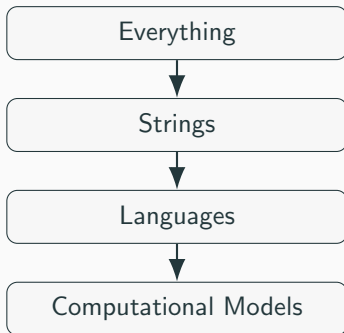
sorting images graphs AI prompts DNA

Why Languages?

Without a common abstraction, these look unrelated:

sorting images graphs AI prompts DNA

With languages:



Which Machines Recognize Which Languages?

Machine model	Memory idea	Language class
Finite Automata	finite memory	Regular Languages
Pushdown Automata	one stack	Context-Free Languages
Turing Machines	unbounded memory	Computable Languages
Efficient Algorithms	bounded resources	Complexity Classes

Which Machines Recognize Which Languages?

Machine model	Memory idea	Language class
Finite Automata	finite memory	Regular Languages
Pushdown Automata	one stack	Context-Free Languages
Turing Machines	unbounded memory	Computable Languages
Efficient Algorithms	bounded resources	Complexity Classes

This is the roadmap of the course.

Mini Assignment

Define each of the following as a language.

1. Valid BUET student IDs.
2. Valid email addresses.
3. Valid JSON objects.
4. Binary strings representing numbers divisible by 5.

Everything can be encoded as strings.

Every computational problem can be viewed as a language.

Theory of Computation asks:

Which machines can recognize which languages?